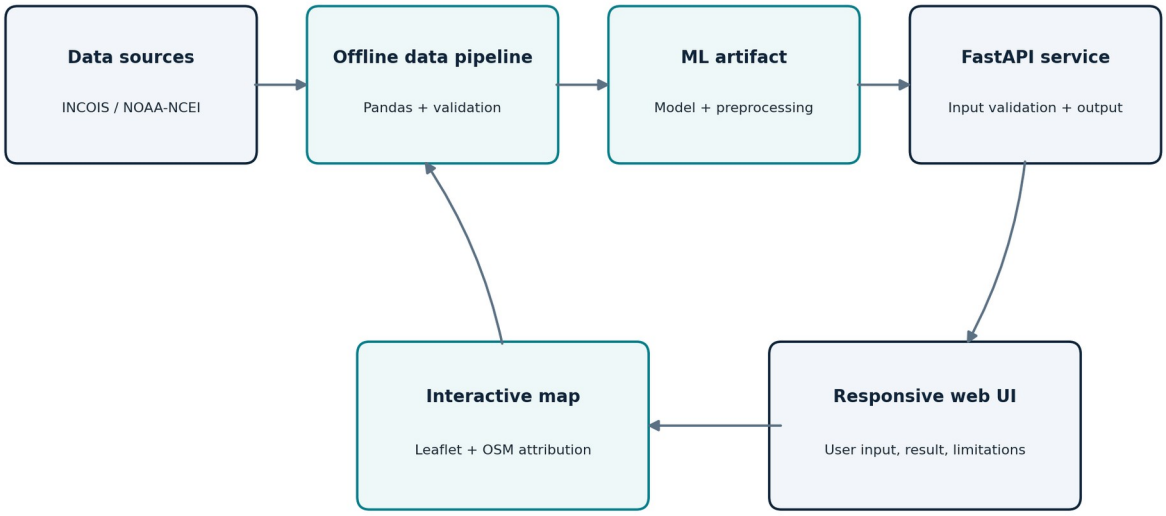
OCEANIX

AI-BASED MARINE INTELLIGENCE

Fishing-Zone Suitability Prediction and Map-Based Decision Support



Optional later: scheduled ocean-data refresh, user accounts, native Android app

PROJECT BLUEPRINT | VERSION 1.0 | 13 OCTOBER 2026

Prepared as a practical, staged engineering project plan

1. Executive Summary

OCEANIX is a proposed student engineering project that uses real marine environmental data and machine learning to estimate fishing-zone suitability and present the result in an accessible map-based interface. Its purpose is to demonstrate a complete, defensible data-to-application workflow: acquire data, verify it, prepare features, train and evaluate a model, serve predictions through an API, and explain the output to a user.

The project is inspired by the general challenge of helping users interpret marine conditions, not by copying the full SIH ORCA system. OCEANIX will remain intentionally small: one verified prediction task, one reproducible ML pipeline, one API, and one responsive interface.

Core constraint: The dataset audit comes before model development. A current list of PFZ points is useful for displaying advisory locations, but by itself it is not a complete supervised-learning dataset: it does not establish negative examples, matched environmental features, or actual fish abundance.

Project at a glance

Item	Current blueprint
Project name	OCEANIX
Working title	AI-Based Marine Intelligence for Fishing-Zone Suitability Prediction
Domain	Machine Learning, geospatial data, marine/ocean analytics
Primary study area	Prefer Indian coast, initially Karnataka if adequate historical labels and environmental inputs are accessible
Preferred data route	Historical INCOIS PFZ advisories matched with real environmental data, subject to feasibility
Fallback route	NOAA/NCEI accession 0306675: historical skipjack tuna catch/environment data for the Western Indian Ocean; this changes the study scope
MVP delivery	Documented dataset, evaluated model if target is defensible, FastAPI endpoint, responsive map-based web prototype, tests and report
Out of scope initially	Multi-agent AI, autonomous data pipelines, production mobile app, live operational fishing forecasts, user accounts, paid APIs

The project decision rule

1. Use the Indian-coast route only if historical PFZ labels can be accessed in usable form and matched to suitable oceanographic features with enough data for an honest evaluation.
2. If that route cannot be made defensible within a bounded effort, assess the NOAA/NCEI time-series. The NOAA fallback is a Western Indian Ocean skipjack-tuna task, not a Karnataka-specific PFZ model.
3. If neither task is viable, narrow the project to a transparent environmental suitability analysis; label it as an index or exploratory tool, not a trained fish-catch predictor.

2. Contents and How to Use This Blueprint

- 1. Executive Summary
- 2. Contents and How to Use This Blueprint
- 3. Background and Motivation
- 4. Problem Statement and Objectives
- 5. Scope, Assumptions and Non-Goals
- 6. Data Strategy and Source Assessment
- 7. Dataset Audit and Evidence Rules
- 8. Prediction Task, Labels and Data Schema
- 9. Data Processing and Feature Engineering
- 10. Machine-Learning Plan
- 11. Evaluation, Interpretability and Limitations
- 12. System Architecture
- 13. API and Application Behaviour
- 14. User Interface and Map Design
- 15. Repository Structure and Development Setup
- 16. Testing and Quality Assurance
- 17. Implementation Schedule and Deliverables
- 18. Risks, Ethics and Responsible Use
- 19. Acceptance Criteria and Expected Outcomes
- 20. Demo Plan and Viva Preparation
- 21. Glossary
- 22. References and Official Resources

This document is both a design brief and a build checklist. Items marked “must verify” are open decisions, not completed facts. During implementation, update the dataset audit and ML results with real evidence and actual metrics.

3. Background and Motivation

Finding productive fishing areas is challenging because marine conditions vary across space and time. Oceanographic measurements such as sea surface temperature (SST) and chlorophyll-a concentration can provide useful context for identifying conditions associated with potential fish aggregation. However, environmental variables are not fish counts: their relationship to fish distribution depends on species, season, local ecosystem, fishing effort and many other factors.

INCOIS describes Potential Fishing Zone (PFZ) advisories as a proxy for potential shoals of fish aggregation and states that its advisories use satellite-derived SST and chlorophyll information. The operational advisories include coordinates and contextual information such as depth, distance and direction from coastal reference points. PFZ zones can shift because ocean conditions are dynamic. [1]

OCEANIX applies a student-project lens to this space: collect a limited, well-understood dataset; select a prediction question the data can support; evaluate the model on unseen observations; and expose the output through a clear user interface. The value is not a claim that a student model is superior to official science services. It is an educational decision-support prototype that makes the data and its limits understandable.

Who could use a prototype?

User / stakeholder	Need	How OCEANIX may help
Fisher or fishing-support worker	Understand where a model estimates favourable conditions	Map display, location/date, plain-language result and limits
Student / reviewer	See a complete ML engineering workflow	Reproducible data audit, metrics, source links and demo
Marine-data learner	Explore how environment and fishery labels relate	Transparent features, source provenance and simple explanations
Not an intended authority	Official advisories and safety decisions	OCEANIX must not replace official advisories, safety notices or local expertise

Guiding principles

- Evidence before algorithms: inspect the downloaded data before designing a target.
- Honest terminology: “suitability estimate” or “PFZ-like label” when that is all the labels support.
- Reproducibility: fixed data versions, recorded preprocessing, saved model and tests.
- Useful simplicity: one strong baseline and one justified model are better than many unvalidated models.
- Clear limitations: display model date, study region, data coverage and uncertainty caveats.

4. Problem Statement and Objectives

Problem statement

Marine environmental information and fishing-zone advisories may be distributed across sources and formats, while interpreting their spatial and temporal context can require technical knowledge. OCEANIX proposes to combine a verified set of environmental features with an appropriately defined historical target, train and evaluate a machine-learning model, and present its estimate through an understandable map-based interface. The system is intended to support interpretation and learning; it cannot guarantee fish presence, catch volume or safe sea conditions.

General objective

To design and evaluate a practical ML-based marine intelligence prototype that uses real environmental and/or fishery data to estimate a clearly defined fishing-zone-related target and display the result with provenance and limitations.

Specific objectives

- Investigate official oceanographic and fishery data sources and document coverage, units, licensing and accessibility.
- Define one target that is genuinely represented by the available labels or measurements.
- Build a reproducible cleaning, alignment and feature-preparation pipeline.
- Train a simple baseline and compare it with a suitable scikit-learn model when sample size and label quality justify the comparison.
- Evaluate generalisation with a split strategy that considers time and geography, rather than reporting only training performance.
- Expose the fitted pipeline through a validated API without retraining during each request.
- Design a responsive web interface with accessible input controls and, when valid coordinates exist, an interactive map.
- Provide documentation, tests, actual metrics, source citations, limitations and a reproducible demonstration.

Research questions

- Can accessible historical data support a defensible prediction target in the chosen study region?
- Which environmental variables are available and consistently aligned to the target?
- Does a trained model outperform a simple baseline on unseen time periods or locations?
- Can the output be communicated without implying certainty about actual fish presence or catch?

5. Scope, Assumptions and Non-Goals

In scope for the MVP	Not in scope for the MVP
One regional/study dataset and one target definition	Global, all-species, all-season modelling
Data audit and reproducible preprocessing	Unverified web scraping at scale or bulk data ingestion infrastructure
Baseline + one justified model	Deep learning or multi-agent architecture without evidence of need
Offline/history-based prediction with clear coverage	Live operational forecasts unless a reliable live-data feed is explicitly implemented
FastAPI endpoint and responsive browser UI	Production-grade mobile app or app-store release
Map view when coordinates are valid	Marine navigation, route planning, collision avoidance, or safety guarantees
Small automated tests and a readable report	Authentication, user profiles, payments, complex database and cloud hosting

Assumptions to check, not blindly accept

- The project may start with Karnataka because a Karnataka PFZ advisory file has been reviewed, but the final study region depends on historical data and matching environmental observations.
- SST and chlorophyll-a are candidate predictors, not guaranteed fields in every selected dataset. Other variables should be included only if present, documented and scientifically justified.
- The PFZ advisory spreadsheet is an example of operational advisory output; it is not by itself evidence of actual catch or a complete historical ML dataset.
- The prototype may use a static, locally bundled dataset for reproducibility. “Live” must not be used in labels unless the app retrieves current data.
- The ML target, class definition, sample count and final metrics remain open until the dataset audit and initial experiment are complete.

Terms the project should use carefully

Use when supported	Avoid unless specifically measured
PFZ-like advisory pattern / suitability estimate	Actual fish abundance
Model probability for a defined target class	Probability that a boat will catch fish
Historical-data prototype	Real-time operational advisory
Potential suitability / decision support	Guaranteed fishing location, safe route, guaranteed savings

6. Data Strategy and Source Assessment

Dataset choice determines the honest scope of OCEANIX. The following is the recommended decision order, based on official source descriptions available when this blueprint was prepared. It is not proof that every historical file is directly downloadable or already aligned.

Route A - Indian coast using historical INCOIS PFZ labels

INCOIS provides operational PFZ advisories and describes the use of SST and chlorophyll products to identify potential fish aggregation zones. Its data holdings page lists PFZ advisories from 2003 onward in ASCII/JPEG formats. This supports the relevance of investigating historical advisories, but access format and historical bulk retrieval must be tested before committing to training. [1][2]

- Advantage: better alignment with an Indian coastal use case and the Karnataka advisory already reviewed.
- Potential target: whether a location/time cell was included in an historical PFZ advisory; this represents an advisory label, not a direct observation that fish were absent or present.
- Required join: PFZ advisory location/date plus environmental values for the same or justifiably matched spatial/temporal window.
- Hard requirement: define defensible comparison points if classification is attempted. “Not listed in PFZ” must not automatically be called “no fish.”
- Go/no-go test: obtain enough historical records in a machine-readable or reliably parseable form, match features, inspect coverage and document limitations.

Route B - NOAA/NCEI accession 0306675

NOAA/NCEI metadata describes the archive as “Weather, Ocean, and Tuna biomass time-series from 2005-2019 in the Western Indian Ocean.” It contains historical catch data obtained from IOTC and environmental variables sourced from Copernicus Marine Service. The metadata lists SST, sea-surface salinity, mixed-layer depth, sea-surface height, net primary productivity, chlorophyll-a, coordinates and month; two text files contain the compiled catch and environmental time series. [3][4]

- Advantage: the archive is described as combining real historical fishery and environmental time-series.
- Key limitation: the study area is the Western Indian Ocean and the target relates to skipjack tuna. It is not automatically a Karnataka-coast PFZ predictor.
- Before choosing it: download and inspect the actual text files, determine the target field, unit of observation, sample count, missingness, spatial coverage and terms/attribution.
- The model should be described specifically as a Western Indian Ocean skipjack-related task if this route is selected. Do not present its output as validated for Karnataka.

Environmental data route / access investigation

INCOIS maintains an ERDDAP server intended to make oceanographic datasets available in common formats, and its ocean-colour products page describes products such as chlorophyll, SST and related variables. Dataset catalogues, actual variables, date coverage, spatial resolution and download access must be checked dataset-by-dataset. [5][6]

Dataset decision matrix

Criterion	INCOIS historical PFZ route	NOAA/NCEI 0306675 route
Regional fit	Potentially strong for Indian coast; historical archive access must be confirmed	Western Indian Ocean; not India-coast-specific by default
Target meaning	Advisory presence/absence proxy if labels are responsibly constructed	Historical catch/biomass-related target; exact field and unit must be inspected
Environmental fields	SST / chlorophyll are relevant; exact match and historical coverage must be verified	Metadata lists several environmental variables; actual file schema still must be inspected

7. Dataset Audit and Evidence Rules

What has already been reviewed

A Karnataka PFZ forecast spreadsheet was inspected during preparation of this blueprint. It states forecast validity from 4 October 2026 to 5 October 2026 and contains 24 non-empty PFZ location records. The table includes coastal reference name, direction, bearing, distance range, depth range, latitude (DMS) and longitude (DMS). Example landing/coastal reference names include Belekeri, Karwar, Honavar, Malpe and Mulki.

Important interpretation: This file is useful for demonstrating parsing and mapping advisory points. It is a single short-validity operational advisory. It does not contain the environmental feature matrix or a verified negative-class dataset required to train a general supervised classifier by itself.

Mandatory dataset audit checklist

Check	Evidence required	Pass condition
Identity and provenance	Official URL, dataset title, version/download date	Source and origin are documented
Time coverage	Earliest/latest timestamp; time zone/frequency	Enough history for meaningful split and evaluation
Geography	Coordinate bounds, coordinate system, spatial coverage	Matches declared project region and use case
Schema/units	Column names, field definitions, units and encodings	Every predictor and target has a documented meaning
Target validity	How target is measured or constructed	Target corresponds to the claim made in the UI/report
Feature match	Join key, time window, spatial matching method	Predictors align with the target without future information leakage
Sample quality	Row count, duplicates, missingness, impossible values	Quality is quantified and major issues are manageable
Class balance / target range	Class counts or target distribution	Evaluation strategy is appropriate; imbalance is handled honestly
Data rights	License, use and attribution notes	Use follows source terms and citation requirements
Repeatability	Saved raw source files/checksums or clear download instructions	A teammate can reproduce the dataset preparation

Stop/go rule

- GO: source is traceable; target is meaningful; there are enough observations for evaluation; time/location alignment is understood; split and metrics are defensible.
- CONDITIONAL GO: data is usable only for exploratory visualisation or a clearly labelled environmental index. Reduce the claims and keep ML as future work if target labels are absent.
- NO-GO: only a few positive points, no feature match, unknown units, unclear provenance, or a target manufactured from unverified assumptions. Do not train a classifier just to claim AI.

Data lineage record

For every raw dataset, record: source URL, access date, filename, version if listed, date range, geographical bounds, file format, license/attribution, SHA-256 checksum if practical, number of rows, columns, preprocessing script and final output path. Never silently edit raw source files.

8. Prediction Task, Labels and Data Schema

Two possible target designs

Route	Target candidate	What it can honestly claim
A: INCOIS PFZ	Advisory-class label for a defined location/time grid, only if historical PFZ and comparison points can be built defensibly	Estimate similarity to historical PFZ advisory labels in that dataset/region; not actual fish presence
B: NOAA/NCEI	The actual catch/biomass-related numeric target or a documented transformation after file inspection	Estimate that target for the represented Western Indian Ocean skipjack dataset; not Karnataka PFZ
Plan C: no valid label	A rule-based environmental score with explicit rules and provenance, if features support it	An exploratory index, not a trained ML prediction and not validated fish abundance

Proposed conceptual schema

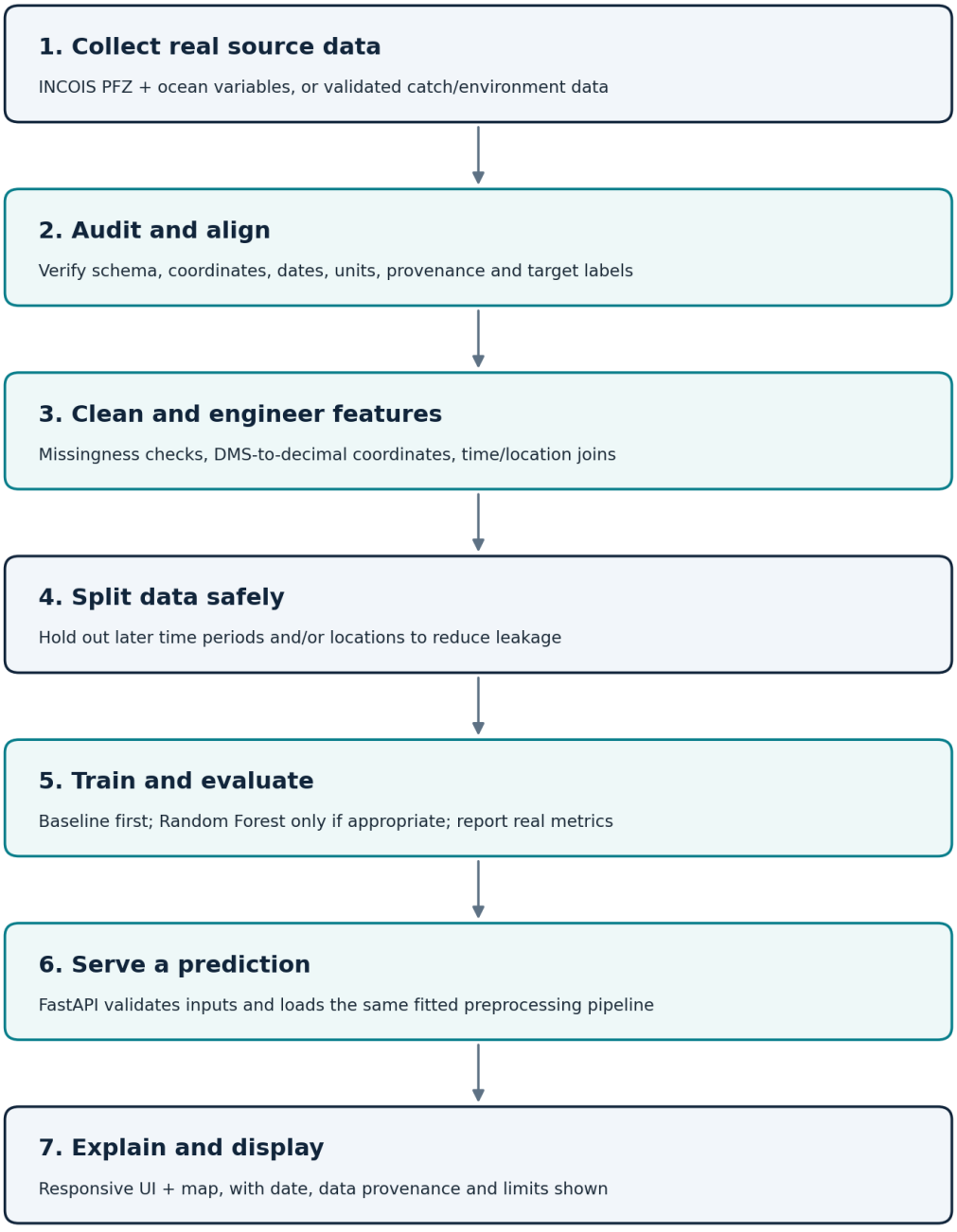
The final schema must reflect the actual file. The following is a design template, not a claim that every column exists:

Field	Type / example	Rule / purpose
observation_id	string or integer	Stable source row/record identifier where available
observation_time	date/time	Record time; preserve source timezone and resolution
latitude	decimal degrees	Convert DMS if needed; validate -90 to 90
longitude	decimal degrees	Convert DMS if needed; normalise to documented convention
sst	numeric, deg C	Only include when present, documented, and time/location aligned
chlorophyll_a	numeric, documented unit	Do not mix unit scales or different product definitions silently
other_environmental_features	dataset-specific	SSS, MLD, SSH, NPP etc. only if genuinely present and justified
target_value / target_label	numeric or class	Store exact target definition and source evidence
target_source	string	Advisory ID / catch source / record reference
data_split_group	date, region or cluster	Support grouped or chronological evaluation without leakage

Target design questions to resolve before coding

- Is the output classification or regression? The type of the actual target decides; do not preselect the answer.
- What does one row represent: point, grid-cell/month, advisory location/day, or fishery reporting unit?
- Can several target rows share the same environmental data because they are from the same date/grid cell? If yes, group them in evaluation.
- If sampling non-PFZ comparison points, what is the sampling rule, and does it represent an unlabelled/no-advisory cell rather than fish absence?
- Will the interface present the model as an advisory-pattern estimate or as catch/biomass prediction? That wording must match the target exactly.

9. Data Processing and Feature Engineering



Processing stages

Stage	Implementation approach	Output / quality check
1. Raw ingestion	Read CSV/TSV/Excel/NetCDF only as needed; keep raw files immutable	Raw file inventory, data dictionary, source notes
2. Schema inspection	Print dtypes, null counts, min/max, unique values, date range and sample rows	Document unexpected fields/encodings before cleaning
3. Coordinate handling	Convert DMS to decimal degrees; validate hemisphere and range; retain original fields if needed	Check converted points on a map and compare sample records
4. Time handling	Parse dates using explicit format; preserve time zone and frequency; do not infer unsupported precision	Report malformed dates, duplicates and ordering
5. Feature alignment	Join by justified spatial/temporal rule; document grid, tolerance and time window	Join coverage and unmatched-row counts
6. Quality checks	Measure missingness, duplicates, impossible values, unit inconsistencies and outliers	No silent row dropping; report exclusion rules
7. Split first	Create grouped/temporal split before fitting data-dependent transformations	No information from final test set enters training transforms
8. Preprocessing pipeline	Use a reproducible scikit-learn Pipeline / ColumnTransformer when suitable	Same transformations at training and serving time
9. Export	Save model, pipeline config, schema and version metadata together	A clean environment can load and predict on a known test example

Specific guidance

- Do not impute a missing target. Remove or investigate rows without valid target evidence.
- For numeric predictors, consider median imputation only if the missingness is understood; add missingness flags only if justified.
- Scaling is usually unnecessary for tree-based Random Forests, but may be needed for models such as logistic regression or distance-based methods.
- Do not fill unknown values with arbitrary zeroes; zero temperature/chlorophyll may have real meaning or be impossible depending on variable.
- Record every filter, conversion and join in code. Save an audit log with row counts before and after each major step.

10. Machine-Learning Plan

Recommended modelling sequence

Stage	Model / method	Reason
Baseline	DummyClassifier / DummyRegressor or a simple domain-appropriate reference	Sets a minimum performance bar; catches models that do no better than trivial guesses
Primary candidate	RandomForestClassifier or RandomForestRegressor, chosen after target inspection	Can model non-linear feature interactions; robust starting point for tabular data
Optional comparison	Logistic/linear regression or HistGradientBoosting if data shape and time allow	Include only if it answers a real comparison question
Interpretability	Permutation importance and/or model-native feature importance; SHAP is optional	Gives a useful model-level explanation without adding unnecessary dependencies

Classification versus regression

Classification is appropriate only if the target is a valid discrete class (for example, a documented PFZ-advisory label). Regression is appropriate only if the target is numeric (for example, a measured catch/biomass value with known units). Do not bin a continuous target into “low/medium/high” merely to simplify the UI unless the bins have a defensible purpose and are documented.

Training and validation protocol

- Create a frozen test set based on later time periods and/or held-out spatial groups when practical. Choose the split strategy from the sampling design, not convenience.
- Avoid random row splitting when nearby coordinates, repeated observations, or same-date environmental grids can leak similar data across train and test.
- Tune using only training/validation data. Evaluate the final locked model on the test set once after model decisions are complete.
- Set random seeds for reproducibility where supported. Record Python and package versions.
- Report sample counts by split and class/target distribution for every experiment.
- Use cross-validation appropriate to group or time structure if enough independent groups exist; do not force k-fold CV on a tiny dataset.

Implementation sketch (conceptual)

1) Load the audited feature table. 2) Define X using documented predictor columns and y using the audited target. 3) Build a preprocessing + estimator pipeline. 4) Split by time/location groups. 5) Fit the baseline. 6) Fit one justified model. 7) Calculate metrics and plots. 8) Save the complete pipeline and model metadata. 9) Run a reload-and-predict smoke test. This is a workflow outline; actual code and parameters must be based on the real data.

Feature leakage examples to avoid

- Using future dates or future environmental measurements to predict an earlier PFZ advisory.
- Using coordinates that directly encode the advisory class because the target construction makes the location itself a label shortcut, without testing whether it generalises.
- Joining target records to the nearest environmental row without limiting temporal distance.
- Scaling, selecting features or imputing using the complete dataset before train/test separation.
- Keeping duplicate or adjacent observations in both train and test when they are essentially the same event.

11. Evaluation, Interpretability and Limitations

Metrics

Task	Minimum reported metrics	Why
Classification	Confusion matrix, precision, recall, F1, balanced accuracy; PR-AUC if classes are imbalanced and probabilities are meaningful	Accuracy can hide poor minority-class performance
Regression	MAE, RMSE and R-squared, plus target distribution and an error plot	MAE is interpretable in target units; RMSE penalises large errors; R-squared alone is not enough
Spatial/temporal generalisation	Performance by held-out time period or geographic group where sample size permits	Shows whether the model works beyond near-duplicate training observations

Required result artefacts

- A dataset summary: row counts, variables, missingness, region/date range and target definition.
- A baseline-versus-model metric table using the same test split.
- A confusion matrix or predicted-versus-actual/error plot appropriate to the target.
- A brief statement of where model performance is weak, including groups or periods with limited data.
- A saved test example showing expected input and actual output from the exported pipeline.
- A clear “not a catch guarantee” notice in both the report and user interface.

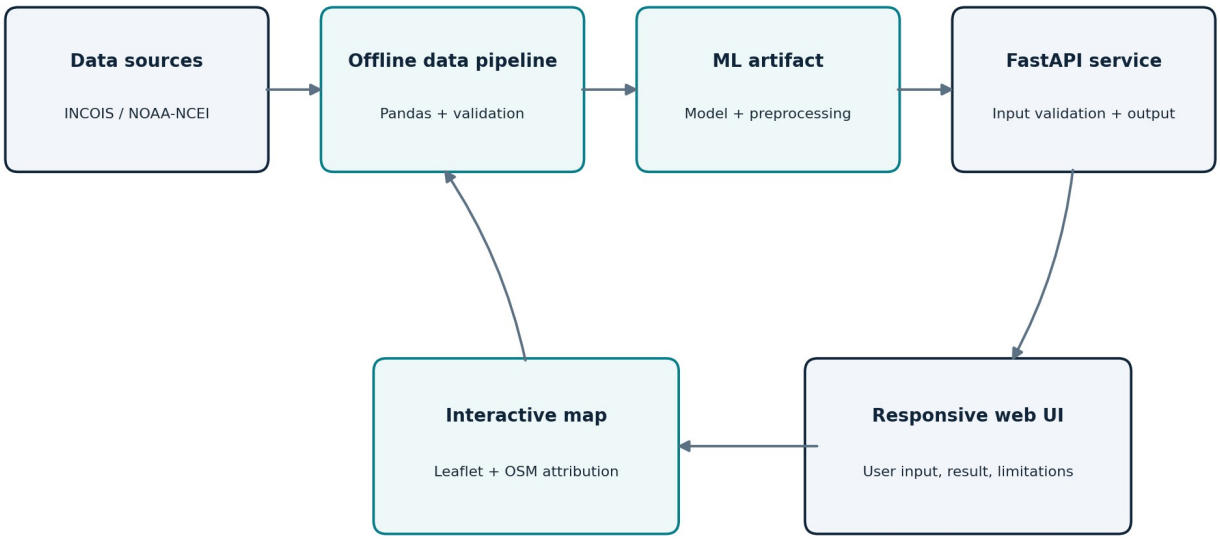
Interpreting feature importance

Feature importance indicates which input variables helped the fitted model on the evaluated dataset; it does not establish causation. Correlated predictors can divide or distort importance. Explanations should be phrased as model behaviour (for example, “SST contributed strongly to this model’s decisions”) rather than oceanographic proof that SST causes fish aggregation.

Limitations to state publicly

- Marine systems are dynamic, so an estimate based on historical data may not transfer to a different season, species, region or year.
- PFZ advisory labels are a proxy for potential aggregation, not direct observation of fish presence, effort, catch, or economic outcome.
- If the sample set is small or concentrated in a region/time, the model may not generalise beyond it.
- Cloud cover, sensor coverage, missing environmental observations and differences across products can affect the available inputs.
- The model does not incorporate all environmental, ecological, fishing-effort, regulatory or safety variables.
- The prototype is not an operational navigation system or official advisory service.

12. System Architecture



Optional later: scheduled ocean-data refresh, user accounts, native Android app

Component responsibilities

Component	Responsibility	MVP implementation
Raw data store	Preserve downloaded source files without modification	Local `data/raw/`; only files permitted for use
Preparation scripts	Parse, validate, align and document rows	Python scripts / notebooks; deterministic outputs
Model pipeline	Fit preprocessing and estimator; save artifact and metadata	scikit-learn Pipeline saved with joblib/pickle-style trusted local artifact handling
API layer	Validate requests, load artifact and return response	FastAPI; local development server
Frontend	Collect supported inputs, show result/provenance/limitations	Responsive HTML/CSS/JS or a lightweight chosen framework
Map	Plot only valid locations and labels, with attribution	Leaflet + suitable map tiles; OpenStreetMap attribution when its tiles are used
Testing/docs	Check predictable outputs and reproducible setup	pytest / unit tests / README / reports

Design principles

- Separate model training from inference. The API should load a fixed artifact and not retrain for each request.
- Keep source data, processed data, model artifacts, documentation and UI code in separate folders.
- The API returns the target type, units/class definition, data context, model version and limitation text where practical.
- The user interface must not display example markers as real predictions. Demo fixtures are labelled “sample/demo only.”
- Avoid cloud hosting until the local version works; local development avoids unnecessary hosting charges and credentials.

13. API and Application Behaviour

Minimal API contract

The exact feature schema is defined after dataset audit. The examples below describe the contract shape, not final field names or numeric values.

Endpoint	Method	Purpose	Example behaviour
/health	GET	Confirm API and model are available	Returns status and model version; no user data required
/model-info	GET	Explain supported target, region, feature schema and limitations	Returns metadata from the saved model config
/predict	POST	Validate supported inputs and return estimate	Reject missing required features, impossible coordinates and unsupported schema

Request/response expectations

Request should contain the verified input fields and, when necessary, a date/location identifier that allows features to be retrieved from the project's local data store. Response should include the estimate, target name/type, probability or numeric value only if appropriate, model version, input summary, timestamp, and a brief limitation statement. Do not return a made-up confidence score if the model does not provide a calibrated probability.

API safeguards

- Use request validation through typed schemas (e.g. Pydantic models) and meaningful 4xx errors for invalid requests.
- Do not silently accept unknown feature names or arbitrary units.
- Handle missing model artifacts and incompatible model versions with a clear service error.
- Do not write user-submitted data or secrets to logs unless needed and approved.
- Keep CORS restricted to the local frontend origin during development.
- Use a single model loader and reuse the loaded pipeline; do not reload the model on every prediction unless required.

Example conceptual response

```
{
 "status": "ok",
 "target": "<verified target name>",
 "prediction": "<model output>",
 "model_version": "<version or commit>",
 "study_region": "<verified study region>",
 "limitations": "Historical-data estimate; not a guarantee of fish presence or catch."
}
```

14. User Interface and Map Design

The recommended MVP is a responsive web app that can be opened on a laptop or mobile browser. Figma can be used to design the screens first. A native Android implementation is optional future scope; it should not block validation of the core data and model.

Primary screens

Screen	Elements	Behaviour
1. Home / overview	OCEANIX name, short purpose, study region and data freshness	Explains what the system does and what it does not do
2. Prediction input	Date/location selector or supported feature fields; only the model's real schema	Validates inputs; no fields are invented for visual effect
3. Prediction result	Class/suitability estimate or regression result; model/source/date context	Shows label/value and target meaning in plain language
4. Map	Valid coordinates, marker popup, legend, zoom, source attribution	Only maps genuine points; distinguish observed advisories from model predictions
5. About / data & limitations	Source links, methodology, coverage, model version and warnings	Builds trust and supports reviewer questions

Recommended map legend

If historical advisories are shown as data points, label them "historical INCOIS advisory locations." If model predictions are shown, label them "OCEANIX model estimates." Never use the same marker or colour for both without a visible distinction. If the model output is a class, describe the class label and the training definition; if it is a numeric catch target, include units and the target's spatial/time scale.

UX and accessibility checklist

- Mobile-first layout with readable font sizes and high contrast.
- Visible loading state when requesting predictions and readable error messages.
- A "clear inputs" or reset action.
- Do not require sign-in for the local MVP.
- Display date, location, model version, and source context where relevant.
- Provide a text alternative to the map result for users who cannot interpret colours or maps.
- Show a disclaimer near the result, not hidden only in documentation.
- Use OSM attribution if OpenStreetMap tiles are used and follow the selected tile provider's usage policy. [7]

15. Repository Structure and Development Setup

Suggested repository layout

```
OCEANIX/
 README.md
 requirements.txt
 .gitignore
 data/
  raw/          # source files; do not edit in place
  interim/      # temporary cleaned/joined data
  processed/    # reproducible model-ready table
 docs/
  dataset_audit.md
  data_sources.md
  project_scope.md
  ml_plan.md
  ml_results.md
  limitations.md
 notebooks/     # optional exploration, not production logic
 src/oceanix/
  data/         # download/read/validate/align routines
  features/     # preprocessing and feature schema
  modeling/     # train/evaluate/export scripts
  api/          # FastAPI app and schemas
 frontend/      # responsive UI and map
 models/        # trained pipeline + metadata; avoid committing large data
 tests/
  test_data_checks.py
  test_model_smoke.py
  test_api.py
```

Environment plan

- Use a project-specific Python virtual environment. Keep Python and package versions recorded.
- Install only the packages needed for the selected route: pandas, numpy, scikit-learn, joblib, fastapi, uvicorn and a test runner; add geospatial/NetCDF libraries only if actual formats require them.
- Use Git commits at the end of every milestone. Keep `.env`, credentials, large downloads and temporary outputs out of public repositories where appropriate.
- Do not add an LLM API as a dependency. A language model is not required for prediction, and using one could introduce cost, latency and unverifiable explanations.

Local run sequence (planned)

4. Create and activate a virtual environment; install pinned requirements.
5. Run dataset audit/preprocessing script and inspect the resulting report.
6. Train/evaluate model only after target and split are approved; export model plus metadata.
7. Start FastAPI locally and check `/health` and `/model-info`.
8. Start frontend and connect it to local API; test one valid and several invalid inputs.
9. Run automated tests and follow README steps from a clean start to confirm reproducibility.

16. Testing and Quality Assurance

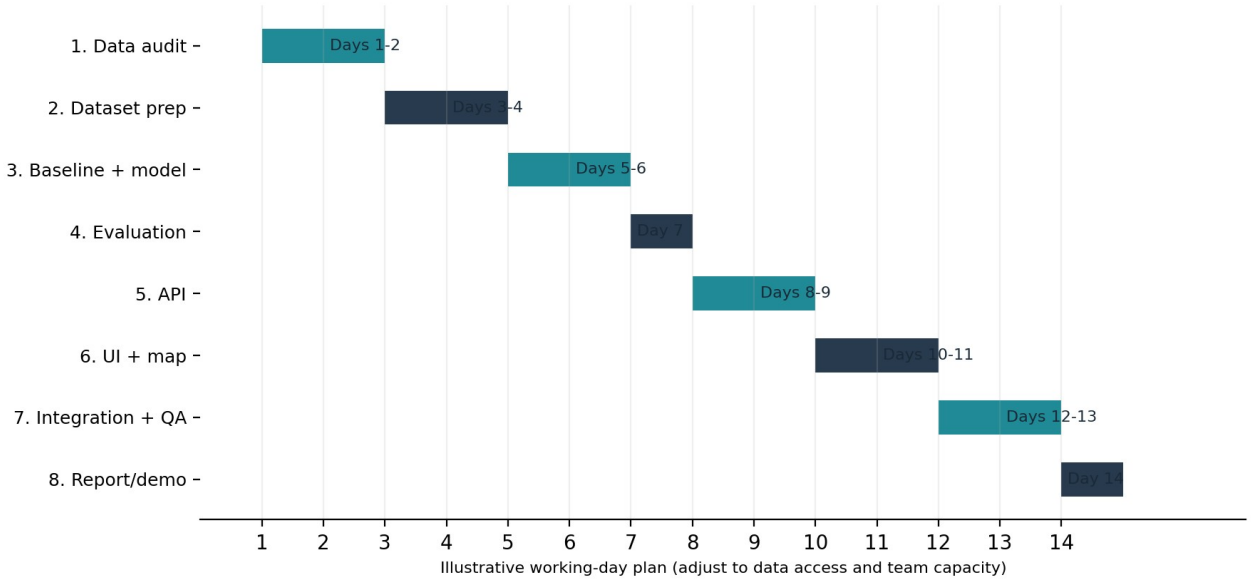
Test category	Examples	Pass condition
Data ingestion	File missing, wrong delimiter, unexpected columns, malformed date	Clear failure with source context; no silent empty dataset
Data quality	Invalid coordinates, duplicates, impossible ranges, missing target	Findings are detected and counted; exclusions documented
Feature alignment	No environmental match; time gap above tolerance; duplicate joins	Unmatched rows reported; no many-to-many join surprise
Model	Load saved artifact, known-schema sample, baseline comparison	Inference reproduces after reload; metrics generated from holdout split
Leakage check	Inspect split overlap by date, grid/location/group	No disallowed group overlap across train/test
API	Valid request, missing field, unknown field, invalid coordinate, absent model file	Appropriate success/error response without traceback leak
Frontend	Loading, empty form, invalid input, API down, result view on mobile	User sees clear status and no fabricated success
Map	Invalid coordinate, marker popup, source attribution, label differentiation	Only valid, correctly-labelled points shown
Reproducibility	Follow README from fresh environment	Another teammate can run the documented demo

Definition of done for the MVP

- The chosen data source and target are documented and defensible.
- All preprocessing steps are scripted and repeatable.
- Model metrics are measured on a held-out dataset and compared with a baseline.
- No unverified real-world prediction is presented as actual catch or fish density.
- API and UI can complete the main workflow locally.
- The map distinguishes source observations/advisories from model estimates.
- Critical test cases pass; known failures and limitations are documented.
- Git history, README and project report reflect the state actually delivered.

17. Implementation Schedule and Deliverables

The following is an illustrative 14-working-day schedule for a focused prototype. Data access is the largest uncertainty; if the data route fails, stop and rescope rather than losing days on repeated downloads.



Milestone	Deliverables	Exit condition
M1 - Data audit (Days 1-2)	`dataset_audit.md`, `data_sources.md`, sample inspection	GO / conditional GO / NO-GO decision documented
M2 - Dataset prep (Days 3-4)	Clean table, schema, data quality report, scripts	Rows/target/features align and units are known
M3 - ML (Days 5-6)	Baseline, candidate model, train/evaluation code	Model beats baseline or honest finding documented
M4 - Evaluate (Day 7)	Metric report, plots, limitations	Leakage-safe test strategy and actual results approved
M5 - API (Days 8-9)	FastAPI endpoints and tests	Model loads and valid/invalid requests behave as expected
M6 - UI/map (Days 10-11)	Responsive prototype and Figma screens	Input-to-result workflow works without fake outputs
M7 - Integrate/QA (Days 12-13)	End-to-end checks, bug fixes, documentation	Clean local run and critical tests pass
M8 - Final demo (Day 14)	Report, slides, screenshots, viva sheet	Demo is reproducible and limitations explained

Team work split (if working in a group)

Role	Owns	Avoid
Data lead	Source audit, preprocessing, data dictionary, licensing	Making undocumented assumptions about labels
ML lead	Target decision, split, baseline, model, metrics	Training on all data or reporting only accuracy
App lead	API, frontend, map and error handling	Hard-coding example results as model predictions
Integration/documentation lead	Git integration, tests, README, report/demo	Late merge of untested code; undocumented changes

18. Risks, Ethics and Responsible Use

Risk	Likelihood / impact	Mitigation
Historical PFZ archive inaccessible or not machine-readable	High / High	Time-box research; inspect known sample; fallback to NOAA/NCEI; document decision
No defensible negative samples or target	High / High	Do not train binary classifier; redefine target or use a transparent non-ML index
Geographic mismatch	Medium / High	Declare the chosen region/species; do not claim Karnataka validity from WIO data
Environmental features do not align with target	Medium / High	Define matching windows and report unmatched rows before training
Data leakage inflates evaluation	Medium / High	Use temporal/spatial/group split, split overlap checks and frozen test set
Model has weak generalisation	Medium / Medium	Compare baseline; state limitations; revise scope instead of hiding metrics
Map looks authoritative despite uncertain output	Medium / High	Labels, confidence caveats, source/date and warning near every result
Live-data access breaks or costs money	Medium / Medium	MVP uses local historical data; live updates only as optional later scope
Agent consumes quota on repeated attempts	Medium / Medium	One bounded stage per prompt; stop on blocker; commit each milestone
Dependency / API incompatibility	Medium / Low	Minimal pinned requirements and smoke tests

Ethics and safety guardrails

- OCEANIX is not official marine advice, a navigation tool, or a substitute for weather, cyclone, sea-state, fishing-ban or local safety communications.
- A modelled “low suitability” value does not mean fish are absent; a “high” value does not guarantee catch.
- Do not recommend sailing into dangerous conditions or imply the tool assesses marine safety.
- Do not display sensitive personal information. The MVP does not need fisher accounts, personal location tracking or individual fishing histories.
- Respect data-source licenses and attribution. Use official sources for provenance; do not redistribute data beyond allowed terms.
- Make clear whether each map marker is a historical source point, an environmental observation or a model output.

Data and model governance

Record dataset version, source access date, feature schema, target definition, train/test split strategy, model parameters, software versions, evaluation metrics and known limitations. If the dataset changes or new rows are added, do not assume past metrics still apply. Re-run the audit and evaluation before describing the updated model as validated.

19. Acceptance Criteria and Expected Outcomes

Expected outcomes

- A clear, reproducible data audit and an explicit go/no-go decision on the target.
- A processed dataset with a data dictionary and transparent row-count / missingness checks.
- A baseline and an evaluated model if the target is supported by sufficient data.
- An exportable trained pipeline served by a small API.
- A responsive interface showing inputs, output, study context, source attribution and limitations.
- An interactive map when valid coordinates and a suitable mapping workflow are available.
- A final project report, README, test evidence, screenshots and viva preparation notes.

Acceptance checklist

Requirement	Acceptance evidence	Status at blueprint time
Source and target verified	`docs/dataset_audit.md` and source links	Pending implementation audit
No fabricated data/results	Code, source records, report	Mandatory rule
Model evaluated on unseen data	Metric table + split definition	Pending dataset choice
Beats or compares fairly to baseline	Same test set, comparison table	Pending ML experiment
API supports correct schema	API tests and example requests	Planned
UI renders correct semantics	Demo + screenshot + source/date label	Planned
Limitations visible	UI disclaimer and report section	Required
Reproducible from README	Clean local setup smoke test	Planned

What a successful project does not require

It does not need a deep neural network, an LLM API, continuous live data ingestion, user accounts, cloud hosting, a native Android app or multiple models. It needs a valid problem definition, trustworthy data, an honest evaluation, a working end-to-end prototype and a clear explanation of trade-offs.

20. Demo Plan and Viva Preparation

Recommended 4-minute demo flow

10. Explain the problem and show the selected data source and study region.
11. Show the dataset audit: what the target means, date coverage, key environmental fields and limitations.
12. Show the evaluation results: baseline, final model and unseen-data metrics. Never invent numbers for the demo.
13. Open the interface, select a supported location/date or input set, and generate one prediction from the API.
14. Show the result on the map only if a valid coordinate exists; distinguish source advisory points from model estimates.
15. Open the “About / Limitations” section and explain what the output can and cannot claim.

Likely viva questions and answer direction

Question	Answer direction
What problem does OCEANIX solve?	It helps interpret marine environmental data through a target-specific ML estimate and map-based interface; it is a prototype, not official advice.
Why did you choose this dataset?	Explain its official source, region, time coverage, variables and target; name verified limitations.
What does the model actually predict?	State the exact target and unit/class definition. Distinguish advisory labels from actual catch or fish abundance.
Why Random Forest?	A practical tabular baseline candidate that can learn nonlinear relationships; used only if it compares well to baseline and fits the data.
How do you prevent overfitting/leakage?	Use holdout by time/location/groups, fit preprocessing only on training data and report unseen-data metrics.
Which metrics did you use and why?	Classification: precision, recall, F1/balanced accuracy/PR-AUC as relevant. Regression: MAE/RMSE/R-squared and residual analysis.
Can high suitability guarantee fish or a catch?	No. The model predicts only its defined target and can be wrong; marine conditions and fishing outcomes are more complex.
Why not use deep learning?	Dataset size/format and the scope do not justify complexity; a simpler model is easier to validate and explain.
How is this different from the existing INCOIS PFZ service?	The project is an educational ML and data-visualisation prototype. It does not claim to replace or outperform the official operational service.
Can it predict for any Indian coastal point?	Only if the selected training data and environmental inputs support that area; no extrapolation claim without testing.
What if your model performs poorly?	Report it honestly, inspect data/label quality, compare with baseline and narrow the claim; weak results are useful findings.
What are your future improvements?	More validated historical data, independent/seasonal evaluation, user testing, responsible data refresh, multilingual UI and perhaps a native mobile app.

21. Glossary

Term	Meaning in this project
AI / ML	Artificial intelligence / machine learning; methods that learn a mapping from examples or data.
PFZ	Potential Fishing Zone: an advisory indicating locations considered favourable for potential fish aggregation, not a guarantee of catch.
SST	Sea Surface Temperature.
Chlorophyll-a	Pigment measurement often used as an indicator of phytoplankton/ocean-colour conditions; not a direct fish count.
SSS	Sea Surface Salinity.
MLD	Mixed Layer Depth.
SSH	Sea Surface Height.
NPP	Net Primary Productivity.
Feature / predictor	An input variable used by a model.
Target / label	The value or class the model is trained to predict.
Data leakage	Information from validation/test targets or future data accidentally influences model training, inflating evaluation.
Baseline	A simple reference prediction used to judge whether a more complex model adds value.
Classification	Predicting a discrete class.
Regression	Predicting a numeric quantity.
API	An interface that lets the frontend request predictions from the backend.
FastAPI	Python framework for building APIs.
Leaflet	JavaScript library for interactive maps; external tiles need proper attribution and policy compliance.
Explainability	Methods to describe which variables influenced the fitted model; this does not prove causation.

22. References and Official Resources

The following official sources informed the data strategy and technology choices. Pages and archive contents can change; record the exact file URL, access date and schema when the team downloads data. Accessed for this blueprint on 13 October 2026.

[1] INCOIS - Potential Fishing Zone Advisory
https://www.incois.gov.in/MarineFisheries/PfzAdvisory
Explains PFZ use, SST/chlorophyll inputs, locations, sectors and the advisory meaning as a proxy to potential fish aggregation.

[2] INCOIS - Data Holdings
https://incois.gov.in/site/dataholdings.jsp
Lists PFZ advisories from 2003 onward in ASCII/JPEG and other oceanographic holdings; historical bulk accessibility still must be verified.

[3] NOAA/NCEI - Accession 0306675
https://www.ncei.noaa.gov/archive/archive-management-system/OAS/bin/prd/jquery/accession/details/306675/5
Metadata for the 2005-2019 Western Indian Ocean skipjack tuna/environment time-series and listed environmental variables.

[4] Data.gov catalog record - NOAA dataset
https://catalog.data.gov/dataset/weather-ocean-and-tuna-biomass-time-series-from-2005-2019-in-the-western-indian-ocean-wio-
Alternative catalog metadata and dataset description; inspect actual files before selecting target.

[5] INCOIS ERDDAP
https://erddap.incois.gov.in/erddap/index.html
Oceanographic data server intended to support subset downloads in common formats.

[6] INCOIS - Ocean Colour Products
https://incois.gov.in/site/services/ChloroGIN.jsp
Describes ocean-colour products including chlorophyll and SST and their operational/research context.

[7] Leaflet - Quick Start
https://leafletjs.com/examples/quick-start/
Map setup and OpenStreetMap attribution guidance.

[8] scikit-learn - Getting Started / Model Evaluation
https://scikit-learn.org/stable/getting_started.html
Model training, train/test evaluation and cross-validation concepts.

[9] FastAPI documentation
https://fastapi.tiangolo.com/
API request validation, endpoint design and local application framework.

Final project statement

OCEANIX commitment: Build a small, traceable and scientifically honest prototype. The final target, geographical scope and model choice will be determined by the actual dataset audit—not by a pre-decided promise to predict fish abundance.